

1 Oblivious Transfers

The beginning of this lectures notes discuss the idea of Oblivious Transfers. Rabin's oblivious transfer protocol is as follows:

1. Sender S sends a bit b of information through the OT.
2. With probability $1/2$, the receiver R receives bit b and with probability $1/2$, it receives a $\#$ symbol which represents loss of information.

The **1-2 OT** is another protocol that does the following:

1. Sender S sends two bits b_0, b_1 into the OT channel.
2. Receiver R submits either $s = 0$ or $s = 1$ and receives b_s from the OT.

1.1 Clear Channels

Each of the above gadgets imply a clear channel, that is, a channel where information can be passed successfully. Firstly, for the 1-2 OT we have that the sender can simply put the same bit for both b_0 and b_1 , or that S and R agree to only use b_0 for communication. For Rabin's OT, we have that the sender S can simply send the same information many times, and with overwhelming probability the information is transferred successfully to R .

1.2 1-2 OT $\iff$ Rabin OT

The above two gadgets are equivalent, meaning that given either one of the two, the other can be constructed.

1-2 OT $\implies$ Rabin OT:

Firstly, we will demonstrate that given a 1-2 OT, we can construct a Rabin OT easily. We assume that R and S have a clear channel to communicate with one another with and that both parties are honest. The construction is as follows:

1. S generates a random value r and flips coin c . If $c = 0$, it sets $b_0 = b$ and $b_1 = r$ if $c = 1$, it sets $b_0 = r$ and $b_1 = b$.
2. R guesses either $s = 0$ or $s = 1$ and receives value $v = b/r$.
3. S tells R the coin flip result for c , indicating whether $s = 0/1$ would result in the correct value being received.

Since R has probability $1/2$ of guessing the correct value for s , it simulates the Rabit OT properties.

Rabin OT $\implies$ 1-2 OT:

We demonstrate how to generate a 1-2 OT given a Rabin OT. For sufficient security parameter k , it requires $3k$ rounds of the Rabin OT and is performed as follows:

1. S sends $3k$ bits to R which will be represented as $r_1, \dots, r_{3k}$. The expected number of received bits is $1.5k$ bits for different r_i values and $1.5k$ # symbols.
2. Following Chernoff bounds, the probability that R receives $\geq 2k$ r_i values is negligible and the probability it receives $\leq k$ r_i values is also negligible, so it must have at least k r_i values sent from S .
3. R selects either $s = 0$ or $s = 1$ as follows. It selects two disjoint sets of size k of indices for r_i values such that one of the two is composed entirely of i values where it successfully received r_i . Notice that there is a overwhelming probability that it received less than $2k$ bits, so it can only construct a single subset of length k such that it knows every r_i value. It sends these two vectors of length k as (v_0, v_1) with its selection s being the v_s vector that knows all corresponding r_i values.
4. S constructs, using each received vector $\bigoplus_i r_i \oplus b_0$ for v_0 and $\bigoplus_i r_i \oplus b_1$ for v_1 and sends these values to R . Note that the above notation is saying, for each index in v_s , we xor all r_i values sent to R and then xor that value with the corresponding b_s .
5. R receives two values, say v_0, v_1 . Since it knows all the used r_i values for v_s , it can calculate $b_s = \bigoplus_i r_i \oplus v_s$ to get the correct bit value.

2 Implementing 1-2 OT

The following section introduces constructions for 1-2 OT.

2.1 Honest Receiver Construction

The following construction is easy to break if the receiver is malicious, however, it introduces the main idea of the correct protocol:

1. Firstly, the sender A selects a trapdoor permutation $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ as well as a random string r to B .

2. B generates its selection s by choosing two n -length bitstrings (y_0, y_1) . For its selection s , it generates y_s by choosing random n -length bitstring x_s and computing $y_s = f(x_s)$.
3. A computes $(b_0 \oplus \langle f^{-1}(y_0), r \rangle, b_1 \oplus \langle f^{-1}(y_1), r \rangle)$. These two values are sent to B .
4. Since an honest B can only have the inverse to y_0 or y_1 , it can only correctly compute b_0 or b_1 .

The issue with this scheme is that it is very easy for B to cheat. They simply need to compute both y_0 and y_1 by first calculating the input x_0, x_1 . The following is an extension of this that ensures they cannot do this.

2.2 Coin Flips Into the Well

The following protocol produces random bits dictated by Alice, however, Alice cannot see the result of the coinflips. This means that the randomness used by Bob cannot be determined by Bob, however, Alice does not know that randomness he uses. The protocol is as follows:

1. Bob selects random bit $R_b = r_{1b} \dots r_{nb}$.
2. Alice sends n random bits $R_A = r_{1a} \dots r_{na}$ to Bob.
3. Bob's randomness is $R = r_{1b} \oplus r_{1a}, \dots, r_{nb} \oplus r_{na}$.

This makes it very easy for Bob to cheat, however, so the following protocol is instead used in order to ensure that Bob uses the randomness generated by Alice.

2.2.1 ZK 1-2 OT

To ensure that the receiver (Bob) cannot cheat, the following protocol is used. Only the coin flips into the well protocol is described as the rest follows from the honest protocol. The bits are generated as follows:

1. Bob sends commitments for $r_{b1} \dots r_{bn}$ as $com(r_{b1}), \dots, com(r_{bn})$.
2. Alice sends randomness $r_{a1} \dots r_{an}$ and sends it to Bob.
3. Bob retrieves f, r from Alice and returns y_1, y_2 generated from random bits $r_{b1} \oplus r_{a1}, \dots, r_{bn} \oplus r_{an}$.

Now, Bob has to prove to Alice that he used her bits in order to generate the randomness according to the protocol. To do this, first consider the following statement:

$\exists$ decommitments to $com(r_{1b})...com(r_{nb})$ s.t. y_1, y_2 are generated properly

Consider the witness $decom(r_{1b})...decom(r_{nb})$, that just decommits all the r_b values. The key fact is that this witness can be verified in polynomial time since an algorithm can run to just recompute y_1, y_2 given the r values. This means that the above language is **in NP**! Since this is the case, we can use a ZK proof for a NP complete problem (Ham path, Three Coloring) to confirm that the values y_1, y_2 were generated honestly, which ensures that Bob used the coinflips Alice generated.

3 Secure Computation for N Parties

We can simulate the secure computation among n honest players for any function $f : \{0, 1\}^n \rightarrow \{0, 1\}$. We use the following theorem:

Any polynomially computable function $f : \{0, 1\}^{n \cdot |x|} \rightarrow \{0, 1\}$ can be represented as a polynomial size circuit consisting of only XOR and AND gates.

The above construction involves converting f into a turing machine, the turing machine into a tableau, and the tableau into a circuit.

3.1 Setup

The setup of this involves creating shares of the input. This is done among n players $P_1, \dots, P_n$ by having each player send the other $n - 1$ players a random bit $r_{(j,l)}$. Now, each player's share of each input bit a_i is all sent random bits XORed with all received random bits XORed with their actual input $x_{j,i}$.

3.2 Gate Construction

The gates follow nearly exactly from the 2-party construction. For XOR gates, the XOR of all shares of inputs A and B creates C , in other words:

$$\begin{aligned} A \oplus B &= C \\ (a_1 \oplus a_2 \dots \oplus a_n) \oplus (b_1 \oplus \dots \oplus b_n) &= (c_1 \oplus \dots \oplus c_n) \\ (a_1 \oplus b_1) \oplus \dots \oplus (a_n \oplus b_n) &= C \end{aligned}$$

Therefore, we can just have the players XOR their individual shares to produce shares of the output of an XOR gate.

For AND gates, we can decompose the gates into the following:

$$(a_1 \oplus a_2 \oplus \dots \oplus a_n) \cdot (b_1 \oplus b_2 \oplus \dots \oplus b_n) = \sum_{i=1}^n \sum_{j=1}^n a_i b_j$$

Here we have the same sort of distribution as the two party case. All $a_i b_j$ instances where $i = j$ can be calculated locally for each player. For every other term, players p_i, p_j can use their pairwise communication channel to compute $a_i b_j$ following the 1-2 OT strategy used in the 2 player model.

4 Yao's Garbled Circuits

Garbled circuits involve two parties, a Garbler G and evaluator E . The interaction is executed as follows:

1. Garbler has randomness r over which they generate garble circuit c to create $\hat{c}$ or $G(c) = \hat{c}$ and input $\hat{x} = G(x)$. They send $\hat{c}$ and $\hat{x}$ to evaluator E .
2. Evaluator runs the function $\hat{c}(\hat{x})$ and the output must be equal to $c(x)$. However, the evaluator must not be able to learn anything about the input other than what the output reveals.

The basics of Yao's strategy for accomplishing this while maintaining security under the assumption that only one input is given to the user is as follows.

1. First we assume the entire circuit is composed of gates that take in 2 bits and output a single bit. We also assume the circuit's final output is a single bit.
2. We define each wire as being a 0/1 by pair of keys k_0, k_1 , each key corresponding to the respective bit.
3. These keys are represented as a table, and each gate with inputs a_0, b_1 defines its output as encryptions of k_b using the keys from its input wires.
4. To evaluate a gate, the evaluator decrypts the corresponding entry in the table that is encrypted by the two input keys they got from the previous wire (The table must be permuted randomly which is explained in the next set of notes)

5. The final wire's output is either k_0 or k_1 which the garbler can decrypt as 0/1